Network Security: Threats and Cryptography

Lecture 30

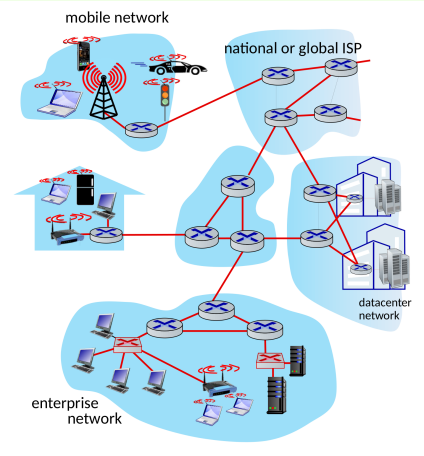
Computer Networks and Distributed Programming

When someone on the network is hostile

- The stack is complete, from the application to the wire, and every protocol on it so far has trusted whoever is at the other end.
- Much of it was designed for a few thousand users who knew each other. Today it carries our credentials, our money and our health records.
- Remember the table of application protocols: next to HTTP, SMTP, Telnet and FTP stood a secured twin. Today, what secured means.
- First: the attacks: what an adversary on the network can actually do.
- Then: the properties a secure communication must have, one for each attack.
- Finally: the cryptography that provides them, from ciphers two thousand years old to the ones in your browser.

Network security

- Network security studies the possible attacks on networks, and the possible ways to prevent them.
- Connectivity is now a utility, like water or power, and a large share of our sensitive data travels on it.
- Attacks differ in purpose and in mechanics, and they evolve with the technology and with who is using it.
- Most combine four moves: explore (the port scans we have seen), listen, impersonate, disrupt.



Malware

Malware is malicious software that reaches a machine through the network: a downloaded file, an e-mail attachment, a vulnerable service. Once inside, it can:

- Pester: show advertisements (adware) or fake alarms that push you to install more (scareware).
- Destroy: delete your files (wiper) or encrypt them and ask for a ransom (ransomware).
- Spy: collect passwords and personal data (spyware); a keylogger records every key you press.
- Take over: obtain administrator privileges and hide itself (rootkit).
- Recruit: turn the device into a zombie, one of many in a botnet that attacks others on command.

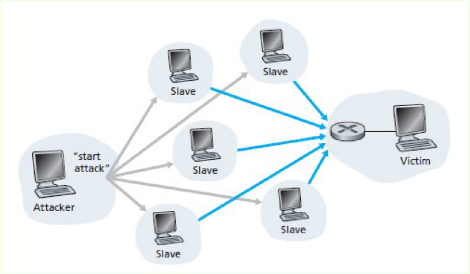
Viruses, worms and trojan horses

- Today's malware is self-replicating: from each infected host it seeks entry into others, and from those into more.
- A virus needs some user interaction: an attachment someone opens, which then mails itself to the victim's contacts.
- It usually arrives disguised as legitimate software: a trojan horse.
- A worm needs no interaction at all. It enters through a vulnerable network application that accepts it unasked.
- Then it scans the network for other hosts running the same application — which is why a worm can spread in minutes.

Denial of service

A DoS attack makes a host, a network or a service (a Web server, DNS) unusable by legitimate users.

- Vulnerability: a few crafted messages that make an application or an operating system stop or crash.
- Bandwidth flooding: so many packets that the legitimate ones cannot get through.
- Connection flooding: half-open or open TCP connections until no more are accepted — the SYN flood.



A distributed DoS (DDoS): the attacker drives a botnet of zombies, and the victim sees traffic from thousands of sources.

Why would a DNS server attack you?

Thousands of DNS servers start sending you replies to queries you never made. Who sent the queries?

An attacker, with your address as the source. A UDP query needs no handshake, so nothing checks it: each server answers the address it was given.

- Reflection: the traffic reaches you from legitimate servers, so there is no botnet to trace and nothing obvious to block.
- Amplification: a small query can bring a large reply: on average about 55 times the bytes for DNS, over 500 for NTP.
- Attacks built this way have reached terabits per second.
- Egress filtering: the defence is at the source: a network drops outgoing packets whose source address is not its own.

Packet sniffing

- A sniffer is a passive receiver that records a copy of the packets it sees, looking for credentials and data: eavesdropping.
- On a broadcast medium, wired or wireless, it just keeps the frames addressed to others instead of discarding them.
- A switch makes it harder, not impossible: we saw how to poison its table. Or malware on a router copies all it forwards.
- It injects nothing into the network, so it is very hard to detect.
- Even encrypted traffic tells a sniffer who talks to whom, and when. We will come back to this with anonymity.
- What must stay private has to be encrypted: it is the only defence that does not depend on the path.

A sniffer you already use

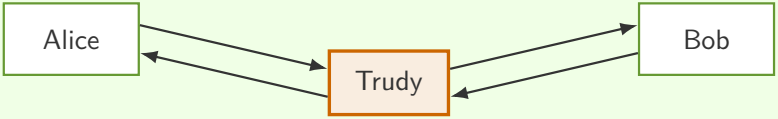
```
$ sudo tshark -i enp3s0 -f 'tcp port 21' -Y ftp        Wireshark, in a terminal
  4 0.000000 192.168.1.17 -> 192.168.1.23 FTP  86 Response: 220 (vsFTPd 3.0.5)
  6 3.210114 192.168.1.23 -> 192.168.1.17 FTP  80 Request: USER ftpuser
  8 3.210706 192.168.1.17 -> 192.168.1.23 FTP 100 Response: 331 Please specify the password.
 10 6.041237 192.168.1.23 -> 192.168.1.17 FTP  81 Request: PASS reti2026
 12 6.088015 192.168.1.17 -> 192.168.1.23 FTP  89 Response: 230 Login successful.
```

- The same Wireshark we have used all along, filtered on the FTP control connection. User name and password, in clear.
- Nothing was sent to anyone: the capture is silent, and neither end can tell it is there.
- Here we sniffed our own traffic. On a shared medium, or with a poisoned switch, it would be anybody's.
- Log in with FTPS or SSH and the same capture shows only encrypted records: this is what the secured twin buys.

Spoofing

- IP spoofing: injecting packets with a forged source address, to look like someone else.
- With an application that trusts certain addresses, it lets the attacker act as a trusted user.
- In a DoS, forged sources are much harder to filter than the addresses of a botnet.
- It is easy wherever the protocol does not check: an Ethernet frame, a UDP datagram, an ARP reply, a DNS answer.
- It is hard in TCP: a forged segment needs the sequence number the other side chose. That is why initial numbers are random.
- The defences are message integrity and end-point authentication: was it modified, and who really sent it?

Man in the middle



- Trudy sits between two hosts: every message from either side goes through her.
- She pretends to be Bob with Alice and Alice with Bob, and each believes it is talking to the other.
- She can read, change, delay or drop anything. We saw one way in: poisoning ARP with the gateway's address.
- Encryption alone does not save them: if the keys were agreed with Trudy, she decrypts, reads, and re-encrypts.

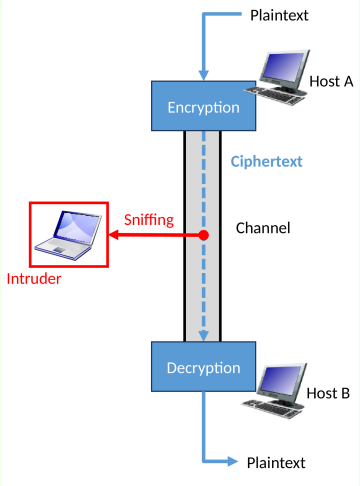
What secure communication must guarantee

Property	What it means	Against
Confidentiality	only the two ends understand the message	sniffing
Message integrity	content unaltered, by malice or accident	modification
End-point authentication	each side can confirm who the other is	spoofing, MitM
Operational security	intruders are kept out of the network	worms, intrusion

- The first three are built into the protocols, and all three rest on cryptography.
- The fourth sits at the edge of an organization's network: firewalls and intrusion detection systems.
- Tanenbaum adds availability, the property a denial of service attacks.

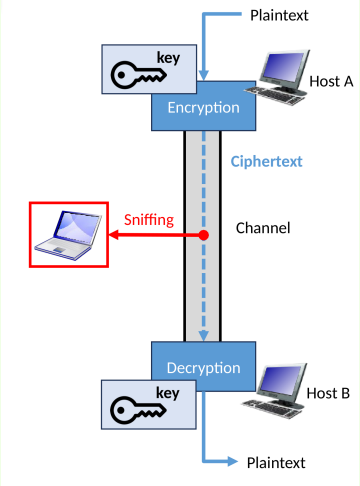
Cryptography

- A cryptographic technique lets the sender disguise data so that an intruder can get no information from it.
- The receiver, and only the receiver, must be able to recover the original.
- Plaintext: the message as written, readable by anyone. The sender encrypts it into ciphertext before it enters the channel.
- A passive intruder only listens: the one in the figure. Encryption is exactly what defeats him.
- An active intruder can also alter, inject and replay messages. Against him, encryption alone is not enough.



The algorithm is public, the key is not

- In every modern system, those of the Internet included, the algorithms are published and standardized. The intruder knows them too.
- What he does not know is the key: a string of bits given to the algorithm together with the message.
- Alice encrypts m with her key K_A and sends $K_A(m)$. Bob decrypts with his key K_B: $K_B(K_A(m)) = m$.
- Symmetric: K_A and K_B are the same key, and it is secret.
- Asymmetric: two different keys: one known to the whole world, one known only to its owner.



Kerckhoffs' principle

Kerckhoffs' principle

All algorithms must be public; only the keys are secret.

- Hiding the algorithm, security by obscurity, never works: sooner or later it leaks, and then everything is exposed.
- An algorithm takes years to design and deploy. A key can be changed in a second, as often as needed.
- A published algorithm is attacked by every cryptographer for free. If it survives years of that, it is probably solid.
- So the secret is all in the key, and its length matters: n bits are 2^n keys to try, and each bit doubles the work.

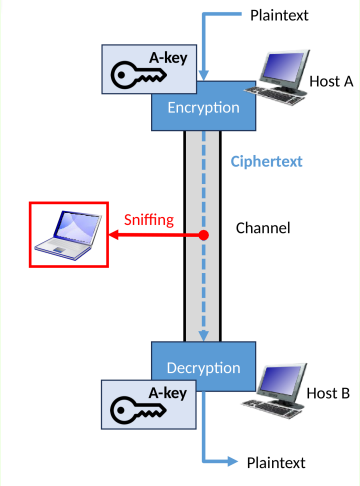
What the cryptanalyst has

Attack	What Trudy has	For example
Ciphertext only	intercepted ciphertext, nothing else	letter frequencies
Known plaintext	some pairs of plaintext and ciphertext	"bob" and "alice" are in it
Chosen plaintext	the ciphertext of plaintext she picks	Alice sends a text Trudy chose

- Each row gives the attacker more than the one above.
- Known plaintext is the normal case, not bad luck: many machines begin every session with the same words, such as `login:`.
- So a cipher is judged by the strongest attack: it must resist even chosen plaintext.

Symmetric key cryptography

- One key, K_S, used both to encrypt and to decrypt. Alice and Bob must both have it, and nobody else.
- Sender: plaintext m and K_S go into the encryption algorithm; the ciphertext $K_S(m)$ can travel on the network.
- Receiver: the ciphertext and the same K_S go into the decryption algorithm, and m comes out.
- This is how cryptography worked for two thousand years, and it still carries almost all encrypted data today.



Substitution ciphers

- Caesar: each letter is replaced by the one k places later: with $k = 3$, `attack` becomes `dwwdfn`. Only 25 keys to try.
- Monoalphabetic: any letter for any other, as long as the mapping is one to one. The key is the whole second row:

plaintext	a b c d e f g h i j k l m n o p q r s t u v w x y z
ciphertext	Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

- $26! \approx 4 \times 10^{26}$ keys: trying them all is hopeless. And `attack` becomes `QZZQEA`.
- Yet it falls to a page of ciphertext. The cipher keeps the statistics of the language: e and t are 13% and 9% of English letters.
- Count, match the frequent letters, then the frequent pairs (`th`, `in`): it has only teaching value, and is never used.

Transposition ciphers

M	E	G	A	B	U	C	K
7	4	5	1	2	8	3	6
p	l	e	a	s	e	t	r
a	n	s	f	e	r	o	n
e	m	i	l	l	i	o	n
d	o	l	l	a	r	s	t
o	m	y	s	w	i	s	s
b	a	n	k	a	c	c	o
u	n	t	s	i	x	t	w
o	t	w	o	a	b	c	d

- The letters are not disguised but reordered: written in rows under the key, read out by columns.
- The key fixes the column order: alphabetical, A first. Column 1 gives `AFLLSKSO`.

Ciphertext

`AFLLSKSOSELAWAIATOOSSCTCLNMOMANT`

`ESILYNTWRNNTSOWDPAEDOBUOERIRICXB`

- Every letter still stands for itself, so the frequencies are those of English: that alone betrays a transposition.
- A guessed phrase such as `milliondollars` then reveals the number of columns, and the rest follows.

One-time pad

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

Example

m	0111
pad	1101
c = m⊕pad	1010
c⊕pad	0111 = m

- The key is a random bit string as long as the message, the pad. The ciphertext is their XOR, bit by bit.
- Bob XORs again with the same pad: XOR is its own inverse, so encrypting and decrypting are one operation.
- Unbreakable, whatever the computing power: for every plaintext of that length there is a pad that produces this ciphertext.
- So the ciphertext carries no information at all about the plaintext.
- It is scrambling at the physical layer again, with a sequence that is truly random and secret.

A pad is used once

What if Alice uses the same pad for two messages?

Trudy XORs the two ciphertexts: $c_1 \oplus c_2 = m_1 \oplus m_2$. The pad cancels out, and the statistics of the language do the rest.

- The pad is as long as the message and must reach Bob secretly. With a channel like that, just send the message.
- One lost or inserted bit and sender and receiver are out of step: everything after it decrypts to garbage.
- It is used only where the pad can travel by hand, on a disk, before any message is sent.
- Stream ciphers: generate a long keystream from a short key. Practical, and bound by the same rule: never reuse it.

First principle: redundancy

An order is 3 encrypted bytes: quantity and product. Trudy has no key and sends random bytes. What does Bob do?

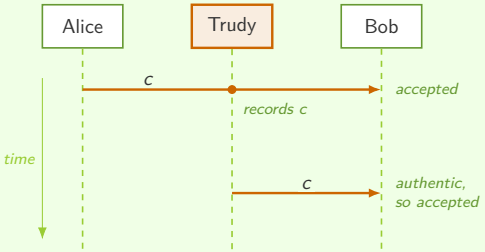
He decrypts them, and almost every 3-byte string is a valid order: he ships 837 swings to a real customer.

- Every encrypted message must contain some redundancy: information not needed to understand it, but needed to check it.
- Alice sends $K_S(m, h)$, with $h = H(m)$ a hash of the message. Bob decrypts and checks that h matches m.
- Random bits pass that check only by chance, with negligible probability.
- Redundancy also helps a passive cryptanalyst recognize a right guess: so never a run of zeros, always something like a hash.

Second principle: freshness

Trudy records c and sends it again tomorrow. Which check fails?

None. c is authentic, so it decrypts and verifies perfectly: a replay attack.



- Put a timestamp inside the encrypted message: redundancy about time, not content.
- Bob rejects anything older than, say, 60 seconds, and remembers what he saw inside that window.
- Or a nonce, a number used once: freshness without synchronized clocks.

Block ciphers

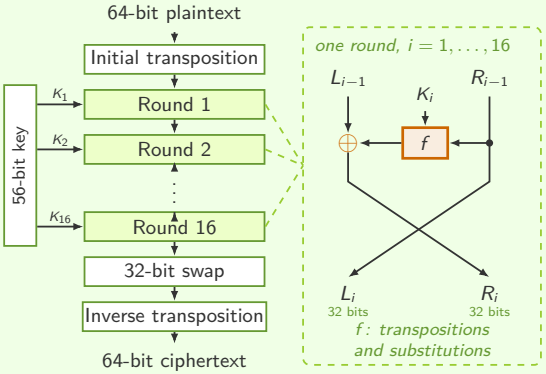
in	out
000	110
001	111
010	101
011	100
100	011
101	010
110	000
111	001

- The message is cut into k-bit blocks, and each block is mapped one to one onto a k-bit block of ciphertext.
- With $k = 3$, the table is the key: `010110001111` becomes `101000111001`.
- A mapping is a permutation of the 2^k inputs: $8! = 40\,320$ keys here, $(2^k)!$ in general.
- With $k = 64$ the table would have 2^{64} rows. So real ciphers compute it: small tables and a bit scrambler, repeated for several rounds.
- The rounds make every input bit affect almost every output bit.

DES and AES

- DES: US standard from 1977: 64-bit blocks, a 56-bit key, 16 rounds. Too short from the start, brute-forced long ago.
- Triple DES: encrypt, decrypt, encrypt with keys K_1, K_2, K_1. With $K_1 = K_2$ it talks to plain DES. Obsolete now.
- AES: chosen in 2001 through an open contest: fifteen public proposals, attacked in public for years before the choice.
- 128-bit blocks, keys of 128, 192 or 256 bits, 10 to 14 rounds, all on whole bytes: fast in software, and built into CPUs.
- A machine that tried all 2^{56} DES keys in one second would need about 149 trillion years for AES-128.

Inside DES: one round, sixteen times



- 19 stages: a transposition, 16 rounds, a swap of the halves, the inverse transposition.
- Each round: the right half passes to the left unchanged; the new right half is $L_{i-1} \oplus f(R_{i-1}, K_i)$.
- f expands R_{i-1} to 48 bits, XORs it with K_i, then substitutes and transposes.
- Each K_i is a different selection of bits of the key.
- Decryption is the same circuit with the keys in reverse order.

Same block in, same block out

Two blocks of plaintext are identical. What does Trudy see?

Two identical blocks of ciphertext. With the table of the block cipher, `010 010 010` is sent as `101 101 101`.

- Encrypting block by block is ECB mode. A block cipher used this way is a monoalphabetic cipher with 128-bit letters.
- The regularities of the plaintext show through in the ciphertext: a repeated `HTTP/1.1` marks the protocol's structure.
- An active Trudy does worse. A payroll file is encrypted record by record, and each bonus is one block.
- Leslie copies the block holding a colleague's large bonus over her own. Without the key, and it decrypts perfectly.

Cipher block chaining

- Before encrypting, each block is XORed with the previous ciphertext block: $c(i) = K_S(m(i) \oplus c(i-1))$.
- The first uses a random initialization vector, $c(0)$, sent in clear. Without the key it is of no use to Trudy.
- Bob decrypts $c(i)$ with K_S and XORs the result with $c(i-1)$, which he already has.

Plaintext 010 010 010, IV 001

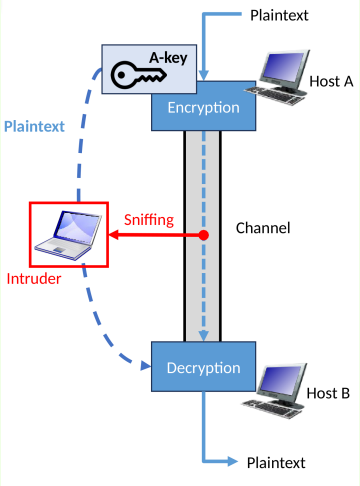
$c(1) = K_S(010 \oplus 001) = K_S(011) = 100$
$c(2) = K_S(010 \oplus 100) = K_S(110) = 000$
$c(3) = K_S(010 \oplus 000) = K_S(010) = 101$

Three equal blocks, three different ciphertexts.

- The cost is one extra block, the IV, and every protocol using CBC must carry it. We will find it inside TLS, up to version 1.2.
- A block pasted from elsewhere garbles two blocks: Leslie is caught.

The key exchange problem

- A symmetric key must be shared before anything is encrypted, and sharing it needs a secure channel.
- Either such a channel exists — the parties meet, a courier — or they need a protocol that makes them converge on a key.
- If the first exchange is intercepted, every later message is readable by whoever caught the key.
- And one key per pair: N users need $N(N-1)/2$ keys, about half a million for a thousand users.



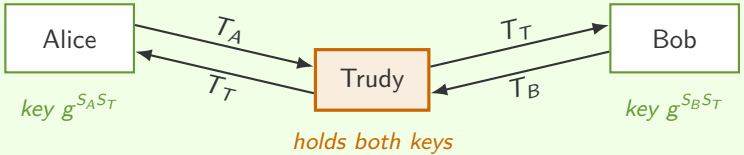
Diffie-Hellman key exchange

- Alice and Bob agree in public on a large prime p and a number g. Trudy knows them too.
- Alice picks a secret S_A and sends $T_A = g^{S_A} \bmod p$. Bob picks S_B and sends $T_B = g^{S_B} \bmod p$.
- Alice computes $T_B^{S_A} \bmod p$, Bob $T_A^{S_B} \bmod p$. Both are $g^{S_A S_B} \bmod p$: the same key.
- Trudy has p, g, T_A and T_B. To get S_A from T_A she needs a discrete logarithm, and no fast method is known.

$p = 47$, $g = 3$, $S_A = 8$, $S_B = 10$

Alice sends $3^8 \bmod 47 = 28$, Bob sends $3^{10} \bmod 47 = 17$.
Alice: $17^8 \bmod 47 = 4$. Bob: $28^{10} \bmod 47 = 4$. The key never crossed the network.

Man in the middle, on Diffie-Hellman



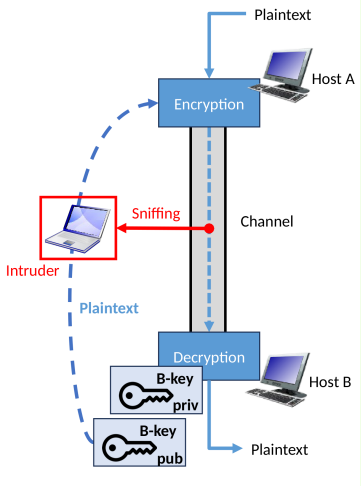
When Bob receives T_A, how does he know it came from Alice?

He does not. Trudy intercepts both values and answers each side with her own T_T: there are two keys, and she has both.

- Alice's messages decrypt at Trudy, are read, changed if she likes, and re-encrypted for Bob. Neither side sees anything wrong.
- Diffie-Hellman agrees on a key but says nothing about with whom. It needs authentication on top.

Public key cryptography

- Bob has two keys: a public one, K_B^+, for the whole world, and a private one, K_B^-, that never leaves him.
- Alice encrypts with K_B^+; only K_B^- undoes it: $K_B^-(K_B^+(m)) = m$. No secret is exchanged.
- Trudy has K_B^+ and can encrypt any m: the cipher must survive a chosen-plaintext attack.
- With RSA the keys also work the other way round, $K_B^+(K_B^-(m)) = m$: the basis of digital signatures.
- But anyone can encrypt for Bob: the ciphertext says nothing about who sent it.



Choosing the keys

1. Choose two large primes p and q; today their product has at least 2048 bits.
2. Compute $n = pq$ and $z = (p-1)(q-1)$.
3. Choose $e < n$ with no common factor with z.
4. Find d such that $ed \bmod z = 1$.
5. Public key $K_B^+ = (n, e)$; private key $K_B^- = (n, d)$.

- Encrypt: a message is a bit pattern, so a number $m < n$: $c = m^e \bmod n$. Decrypt: $m = c^d \bmod n$.
- From n and e one gets d only by factoring n into p and q. No fast way to do it is known.

RSA with small numbers

$p = 5$, $q = 7$: $n = 35$, $z = 24$; $e = 5$, and $d = 29$ because $5 \cdot 29 - 1 = 144 = 6 \cdot 24$.

	m	m^e	c	$c^d \bmod n$
l	12	248832	17	12
o	15	759375	15	15
v	22	5153632	22	22
e	5	3125	10	5

- Each letter is its number in the alphabet, encrypted with $(35, 5)$ and decrypted with $(35, 29)$.
- o and v come out unchanged: with numbers this small RSA is just a substitution cipher.
- Why it works: with $n = pq$, $m^{ed} \bmod n = m^{ed \bmod z} \bmod n = m^1 = m$.
- The same identity read the other way gives $(m^d)^e = m$.

Session keys

- RSA is slow: exponentiations on 2048-bit numbers, where AES does simple operations on 128-bit blocks.
- Used directly, it is also ECB again: the same block always gives the same ciphertext.
- So nobody encrypts data with it. Alice picks a random session key K_S and sends Bob $K_B^+(K_S)$.
- Diffie-Hellman reaches the same point by another road: a shared K_S that never crossed the network.
- From then on the data goes with a fast symmetric cipher, AES with K_S. Public keys start a session; symmetric keys carry it.
- This is exactly how TLS works, and where we are heading.

Certification authorities

- Is K_B^+ really Bob's? If it is Trudy's, Alice encrypts for the attacker: the man in the middle is back.
- A Certification Authority (CA) binds a public key to an identity. It has two jobs.
- Verify: that an entity is who it says it is. No protocol for that: you must trust the CA to have checked properly.
- An unreliable CA is soon trusted by nobody. State and federal CAs offer a reasonable guarantee, but it is still trust.
- Certify: issue a certificate: the public key together with a globally unique identifier of its owner, a name or an IP address.
- Why a certificate cannot simply be forged needs a digital signature: that is where authentication begins.

Message integrity and hash functions

Message integrity, also called message authentication, means that Bob can check two things:

- the message was not tampered with on its way;
- it was really sent by Alice.

As with parity, checksums and CRCs, a short check value goes with the message, computed by a hash function: data of any size in, a fixed-size value out.

Cryptographic hash function

A hash function H for which it is computationally infeasible to find any two different messages $x \neq y$ with $H(x) = H(y)$.

- Today SHA-256: 256 bits of hash. MD5 and SHA-1, still in many books, have been broken.

A checksum is not enough

IOU1	49 4F 55 31
00.9	30 30 2E 39
9BOB	39 42 4F 42
sum	B2 C1 D2 AC

IOU9	49 4F 55 39
00.1	30 30 2E 31
9BOB	39 42 4F 42
sum	B2 C1 D2 AC

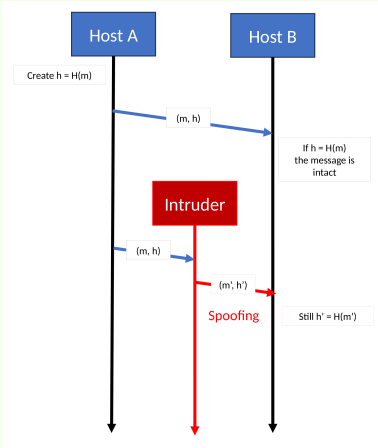
Bob owes Alice $100.99. The IOU becomes $900.19. Does a checksum notice?

No. Adding the bytes four at a time gives the same value for both messages.

- A checksum or a CRC is built against accidents. Nobody designed it against someone who chooses the bits.
- Given a message, finding another with the same check value is easy. For a cryptographic hash it must not be.

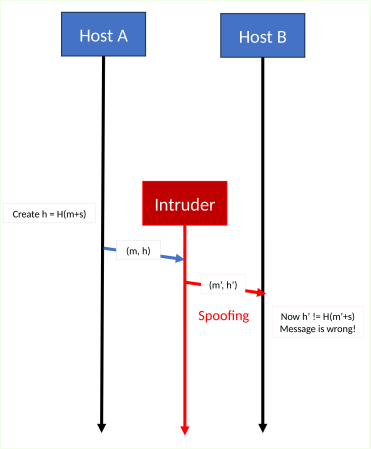
Appending the hash

- Alice computes $h = H(m)$ and sends the pair (m, h). Bob recomputes $H(m)$: if it equals h, the message is intact.
- If Trudy changes only the message, Bob finds $H(m') \neq h$ and discards it.
- But H is public — Kerckhoffs again. Trudy computes $h' = H(m')$ herself and sends (m', h').
- Bob's check passes. The approach is flawed: anyone can produce a consistent pair.
- The hash must depend on something Trudy does not have.



Message authentication code

- A shared secret s, the authentication key. Alice sends $(m, H(m+s))$: $H(m+s)$ is the MAC.
- Bob recomputes it with his s. Without s, Trudy cannot produce $H(m'+s)$: the forgery is caught.
- Nothing is encrypted: m travels in clear. s is shared like a symmetric key, and that is all.
- s must still reach both sides: public keys and certificates.
- A SYN cookie was one: a hash with the server's secret. (Not the MAC address of an adapter.)



Summary

- An attacker can infect hosts, flood them, listen to traffic, forge addresses and sit in the middle.
- A secure communication needs confidentiality, integrity, end-point authentication, and a network that keeps intruders out.
- Algorithms are public, keys are secret. Messages need redundancy and freshness.
- Symmetric: AES (CBC here, GCM in TLS 1.3): fast, but the key must be shared. The one-time pad is perfect and impractical.
- Public key: RSA and Diffie-Hellman set up a session key; a certificate must say whose key it is, or Trudy is in the middle.
- Integrity: a MAC, a hash over the message and a shared secret. Next: signatures, authentication, and TLS, which uses it all.